MASTER OF TECHNOLOGY

ARTIFICIAL INTELLIGENT SYSTEMS

ISS-IRS-PRACTICE MODULE: 2026-27

INTELLIGENT REASONING SYSTEMS

PROJECT REPORT

Predictive Enterprise Project Risk Analysis using Temporal Knowledge Graphs

Group Number: 14

Team:

Akansha Sajimon	A0352134N
Cho Laqshya	A0357002R
Rajamanickam Abirami	A0354513J
Ravikumar Prithika	A0354287U

Contents

1. PROBLEM DESCRIPTION AND BUSINESS CASE 3
1.1 Business problem 3
1.2 Proposed solution and intended value 3
1.3 Scope and measurable objectives 3
2. MARKET RESEARCH AND POSITIONING 4
2.1 Existing solution landscape 4
2.2 Target market and differentiation 4
2.3 Market assumptions and validation needs 5
3. DATA SOURCES, DATASET CONSTRUCTION AND GOVERNANCE 5
3.1 Change in dataset strategy following faculty feedback 5
3.2 Public Jira acquisition and source preservation 5
3.3 Project-instance selection and historical sampling 5
3.4 Grounded PRD and meeting generation 6
4. SYSTEM OVERVIEW AND ARCHITECTURE 6
4.1 System Purpose and Scope 6
4.2 End-to-End Architecture 7
4.3 Design Principles 7
4.4 Scope and Operational Boundaries 7
5. KNOWLEDGE ACQUISITION AND REPRESENTATION 8
5.1 Data Sources and Preparation 8
5.2 Information Extraction and Structured JSON 8
5.3 Entity Resolution 11
5.4 Temporal Knowledge Graph 11
5.5 Knowledge Quality and Traceability 11
6. RISK PREDICTION AND REASONING 12
6.1 Weekly Snapshots and Outcome Labels 12
6.2 Temporal and Graph Feature Engineering 12
6.3 Final Predictive Models 13
6.4 Model Validation and Temporal Generalisation 14
6.5 SHAP Explanations and Investigation Trigger 14
6.6 Agentic GraphRAG Investigation 15
7. SYSTEM DEVELOPMENT AND IMPLEMENTATION 18
7.1 System Development Approach 18
7.2 Implementation of System Components 18
7.3 System Integration and Execution 18
7.4 Agentic GraphRAG Implementation 19
7.5 System Validation and Integrity Checks 20
7.6 Engineering Considerations and Limitations 21
8. EXPERIMENTS AND EVALUATION 22
8.1 Evaluation 22
8.2 Phase 1: Information Extraction and Entity Resolution 24
8.3 Phase 1: Temporal Knowledge Graph Fidelity 24
8.4 Phase 2: Risk Prediction 25
8.5 Phase 3: GraphRAG Retrieval Relevance 26
8.6 Phase 3: Qwen Report and Evidence-Support Evaluation 26
8.7 Additional Retrieval and Faithfulness Experiments 26
8.8 Consolidated Interpretation and Evaluation Boundaries 27
9. FINDINGS AND DISCUSSION 28
References and internal project materials 28

1. PROBLEM DESCRIPTION AND BUSINESS CASE

1.1 Business problem

Enterprise software delivery generates substantial operational evidence across issue trackers, requirements documents and meeting discussions. These artefacts record different views of the same evolving project. Jira captures execution and blockers, product requirements documents (PRDs) record intended scope, and meeting records capture decisions and emerging concerns. However, the links among these records are often difficult to reconstruct, particularly when the relevant events occur at different times.

Conventional status reporting may show the current condition of an individual ticket without explaining how a requirement revision, an unresolved blocker and a downstream dependency jointly increase delivery risk. Project managers must manually reconcile records to investigate such patterns. This introduces delayed escalation, inconsistent interpretation and limited traceability between a warning and its supporting evidence.

1.2 Proposed solution and intended value

The proposed intelligent reasoning system builds a temporal knowledge graph of project artefacts, reconstructs the information available at historical checkpoints, estimates near-term schedule, scope and dependency risks, and retrieves evidence relevant to those warnings. It combines knowledge discovery, supervised risk estimation and graph-guided explanation to support earlier, more transparent investigation. The system is designed to assist project managers rather than replace professional judgment.

Table 1.1 Stakeholder Challenges and Proposed System Benefits

Stakeholder	Pain point	Proposed value
Project manager	Fragmented risk indicators across sources	Prioritised checkpoint-level warnings and traceable investigations
Delivery lead	Difficulty tracing cascading blockers	Connected dependency and event histories
PMO / governance team	Inconsistent audit trail for escalations	Source identifiers, timestamps and evidence-linked reports

1.3 Scope and measurable objectives

The prototype addresses enterprise software project histories represented by Jira issues and associated PRD and meeting artefacts. It predicts three risk categories: Schedule Risk, Scope Risk and Dependency Risk, while retaining decision-related information as supporting project context for investigation. The model evaluates risks over a 14-day future horizon using historical project snapshots and their derived features.

Success is assessed separately at each layer: accuracy of information extraction and graph construction, predictive discrimination of risk labels, and relevance and factual support of retrieved

evidence. A successful pipeline execution is not equivalent to accurate risk prediction or reliable causal explanation.

2. MARKET RESEARCH AND POSITIONING

2.1 Existing solution landscape

The project operates at the intersection of project management, connected organisational knowledge and AI-assisted risk analysis. Existing platforms demonstrate that work-item tracking, dependencies and contextual AI are established product categories. The research opportunity is narrower: evaluating whether a point-in-time temporal representation can support risk prediction and evidence-based investigation across multiple artefact types.

Table 2.1 Comparison of Existing Project Management and AI-Assisted Solutions

Solution category / examples	Established capability	Research differentiation
Project dashboards: Jira, Microsoft Planner	Issue state, delivery schedules, blockers and dependencies	Cross-artefact temporal patterns rather than current-state dashboards alone
Connected context: Atlassian Teamwork Graph and Rovo	Links work items, people and organisational knowledge	Explicit historical checkpoints and feature-based risk modelling
AI risk assistance: Microsoft Dynamics 365 Project Operations	Project metadata analysis and risk assistance	Separate supervised warning model from evidence investigation
Generative retrieval and summarisation	Search and synthesis of connected project information	Chronology checks, provenance and inconclusive outcomes when support is weak

2.2 Target market and differentiation

The initial target customer is a software delivery organisation already maintaining structured issue histories and willing to run a limited pilot. The likely buyer or sponsor is a delivery manager, project management office or engineering operations lead. The initial use case is retrospective and near-term risk intelligence for one delivery team, rather than a full replacement for its existing project management platform.

The differentiator is the combination of temporal reconstruction, predictive scoring and evidence tracing. The knowledge graph links records; the predictive model ranks risk; and the investigation layer examines source support. These functions must remain conceptually distinct. Graph connectivity alone does not demonstrate prediction quality, and retrieved text alone does not establish a causal explanation.

2.3 Market assumptions and validation needs

The market review identifies an opportunity for organisations to connect project histories, risk alerts and source-backed investigations in one workflow. A targeted pilot can demonstrate the value

of the approach by measuring investigation time, alert precision, integration effort, user trust and operational requirements. These measures provide a practical basis for evaluating adoption and further development.

3. DATA SOURCES, DATASET CONSTRUCTION AND GOVERNANCE

3.1 Change in dataset strategy following faculty feedback

The original proposal envisaged generating a fully synthetic enterprise project dataset spanning Jira, PRDs and meeting transcripts. This offered control over scenarios and risk labels but could produce patterns less representative of actual issue-tracking behaviour. In response to the professor's feedback, we adopted a grounded hybrid dataset: public Jira histories provide the factual and temporal foundation, while we generated supplementary PRDs and meeting records based on those histories. The two source types are labelled separately to preserve their origins.

3.2 Public Jira acquisition and source preservation

The data acquisition process retrieves the public Jira archive from Zenodo record 15719919, named "2025-06-23 ThePublicJiraDataset.zip". The archive is saved to Google Drive, checked for ZIP integrity and inspected for its members. The preparation process uses per-repository folders containing all_300_tickets_full.json, with Jira issue details and changelog histories. Original Jira inputs are retained separately from generated artefacts.

A repository may cover several years and multiple release cycles rather than one bounded delivery project. Since samples of 300 tickets can span this wider history, we select time-bounded project instances to represent coherent periods of development activity.

3.3 Project-instance selection and historical sampling

For each repository, the selection process constructs a deterministic stream of Jira events and scans candidate 36-week windows at four-week increments. Candidate windows are scored using the number of distinct active tickets, event richness, number of active weeks and activity continuity. The highest-ranked eligible window is selected, with a second non-overlapping window considered only when independently sufficiently informative. Each selected window is treated as a project instance with its own identifier and timeline.

Weekly checkpoints are generated throughout each instance, even in weeks without a PRD or meeting. These checkpoints provide a consistent temporal sampling framework. The downstream handover describes 30 project instances and 987 unique checkpoints in the final snapshot data. These counts refer to downstream processed artefacts, not to the entire original Zenodo archive.

3.4 Grounded PRD and meeting generation

The document-generation process creates supplementary PRDs and meeting records based on historical Jira activity. Supporting documents are scheduled around project commencement and periods of significant development activity, using event-richness and spacing criteria to determine suitable points in the project timeline. The final dataset contains 30 generated PRDs and 60 meeting

records across 30 project instances. These documents provide additional requirement and discussion context for knowledge extraction and temporal reasoning.

Before each generation request, Jira records are reconstructed as of the intended document date. Tickets not yet created are excluded. Future status changes, comments, changelog entries and resolution information are withheld. Descriptions are reconstructed from the changelog where available, and the ticket status is derived at the cutoff. The language model receives a bounded selection of temporally valid, relevant tickets rather than all 300 complete ticket records. Strict prompts instruct it to write plausible project documents based on the supplied history. A separate hidden manifest records the exact Jira context made available for generations and is reserved for provenance and evaluation, not fed back as ordinary extraction evidence.

This approach grounds the timing and operational context of generated documents in observed Jira activity, while retaining the flexibility to create PRD and meeting artefacts needed for cross-source reasoning. Nevertheless, the prose, decisions and discussion content in generated artefacts remain model-generated and may contain plausible but unverified detail. The public Jira records and generated records therefore require separate source-type and provenance labels.

Table 3.1 Dataset Sources, Characteristics and Purpose

Dataset layer	Origin and purpose	Key limitation
Public Jira issue history	Externally sourced tickets, statuses, comments and changes; factual backbone	Public repositories may be incomplete or heterogeneous
Project-instance windows	Deterministic 36-week selections from repository event history	Selected windows are analytical proxies for projects
Generated PRDs	Historically conditioned requirement narratives	Not authentic enterprise PRD documents
Generated meetings	Historically conditioned discussion narratives	Not authentic meeting transcripts

4. SYSTEM OVERVIEW AND ARCHITECTURE

4.1 System Purpose and Scope

The system supports early identification and investigation of risks in enterprise software projects by integrating historical Jira records, project requirement documents and meeting transcripts into a Temporal Knowledge Graph. It captures project activities, requirements, decisions and dependencies to reconstruct historical project states and identify patterns associated with emerging risks. The system predicts three key risk categories: Schedule Risk, Scope Risk and Dependency Risk, at weekly checkpoints and retrieves relevant historical evidence to support risk investigation and explanation.

4.2 End-to-End Architecture

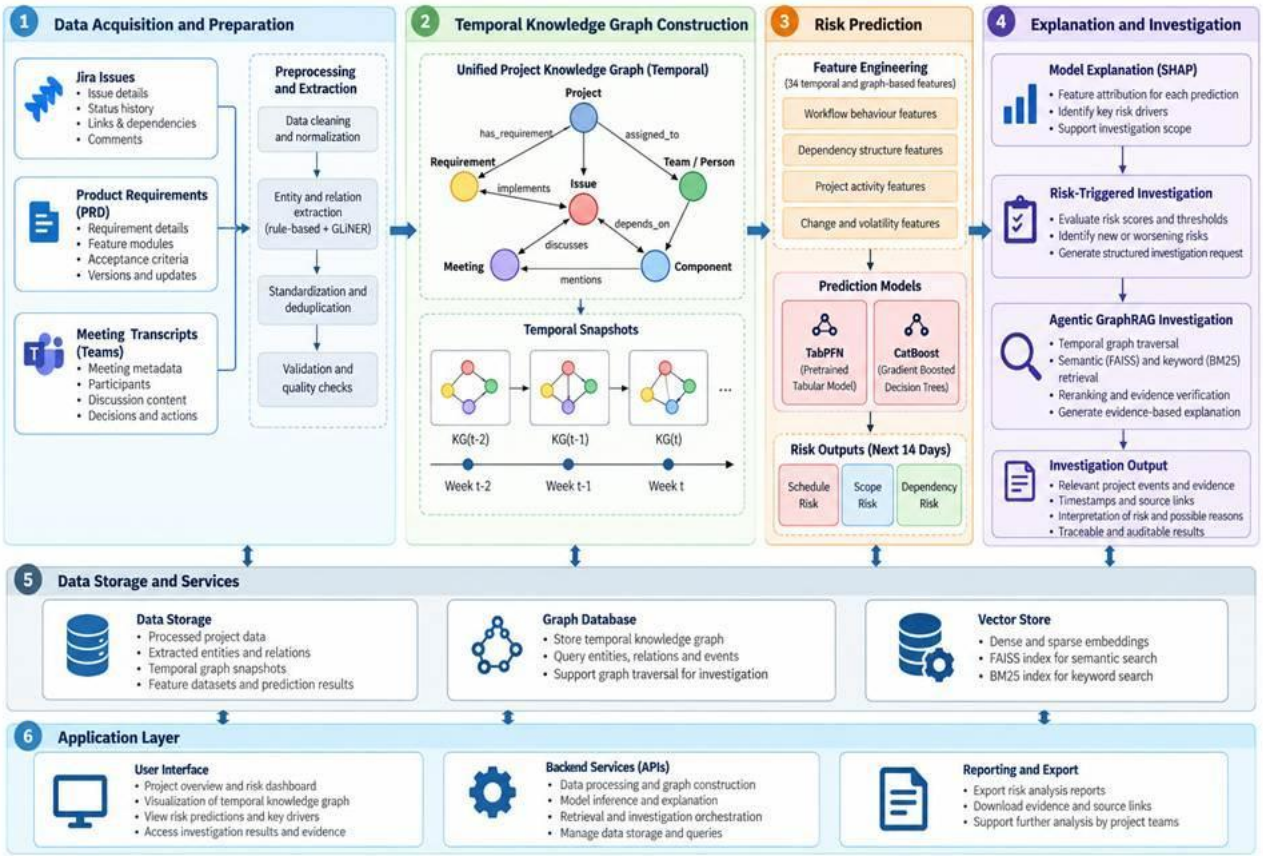


Figure 4.1 End-to-End Architecture of the Implemented Enterprise Project Risk Analysis System

Information moves through explicit intermediate files and schemas, allowing individual stages to be validated independently. The temporal graph is a shared knowledge foundation: it supports both numerical feature generation and the later retrieval of connected evidence. Predictive modelling and evidence-based reasoning therefore have complementary roles rather than duplicating one another.

4.3 Design Principles

Three principles guide the architecture. First, point-in-time correctness separates an event's occurrence from when its evidence became available, so historical predictions cannot rely on later records. Second, traceability links graph facts and investigation statements to source identifiers and supporting text. Third, modularity separates extraction, resolution, graph export, supervised learning and retrieval so that faults and quality limitations can be identified at the relevant stage.

4.4 Scope and Operational Boundaries

The system is developed using a partially synthetic dataset comprising 30 enterprise project instances and 987 weekly project checkpoints. At each checkpoint, the system analyses historical project activities and predicts potential Schedule, Scope, and Dependency Risks over the following 14 days. Temporal snapshots capture changes in project status, requirements, dependencies, and other activities over time, enabling early identification of emerging risks. The system also supports project-

grouped and chronological evaluation to assess predictive performance across different projects and time periods. By combining temporal knowledge representation, machine learning, and evidence-based investigation, the system provides a structured approach to understanding and analysing enterprise project risks.

5. KNOWLEDGE ACQUISITION AND REPRESENTATION

5.1 Data Sources and Preparation

The knowledge-acquisition stage converts the selected Jira histories and supplementary PRD and meeting records into a consistent representation for entity extraction. Jira issue fields, status transitions, comments and dependency links provide structured and historical evidence, while the accompanying documents provide requirement and discussion context.

The extraction processes 5,737 Jira ticket-instance records, 30 generated PRDs and 60 meeting records across 30 project instances. Records are organised in a common JSONL structure containing source metadata, entities, relationships, events, temporal attributes and provenance.

Jira fields and change histories are parsed as structured records, while PRD sections and meeting utterances are prepared for semantic extraction. Source identifiers and timestamps are retained so extracted facts can be traced back to their evidence.

Table 5.1 Tools and Technologies used for Data preparation

Source / tool	Purpose in the implementation
Zenodo public Jira archive	Public, reusable issue records with nested fields and histories
Google Colab + Google Drive	Execution environment and persistent storage
MongoDB restore + PyMongo	Read repository collections and inspect nested Jira documents
Python / pandas	Profiling, selection, transformation, and validation

5.2 Information Extraction and Structured JSON



Figure 5.1 Data preparation, extraction and temporal knowledge representation

The system combines structured-field parsing, regular expressions and GLiNER (urchade/gliner_medium-v2.1) to extract project entities, relationships and events. Explicit Jira fields

and changelogs are processed using deterministic rules, while GLiNER identifies systems, features, components and teams mentioned in free text. The extraction retains source references, timestamps and evidence for later checking.

Extraction is not treated as equivalent to verified truth. Downstream checks distinguish a record found in the source from a successfully extracted, resolved and graph-materialised fact. This is particularly important for meeting narratives, ambiguous names and historical event types that do not map cleanly to the target schema.

The system processes PRD documents and meeting transcripts by dividing the text into smaller, meaningful segments for entity extraction. PRD documents are split based on their sections, with each segment limited to approximately 1,800 characters while retaining its heading and unique identifier. Meeting transcripts are processed at the individual utterance level, preserving speaker information, timestamps, and utterance identifiers. These segments are then passed to GLiNER to identify relevant project entities. The extracted information is stored in a structured JSONL format along with source references, allowing each entity to be traced back to its original document or meeting discussion.

The Jira extractor creates project and issue entities and attaches people and components using explicit Jira fields. Linked issue keys are represented as issue entities with typed relationships such as HAS_ISSUE, ASSIGNED_TO, REPORTED_BY, CREATED_BY, HAS_COMPONENT, DEPENDS_ON and RELATED_TO. Entity identifiers are source-scoped at this stage: they identify occurrences reliably but do not imply that cross-source duplicates have already been resolved.

Historical changes are taken from status history and the full Jira changelog. Each accepted event includes an event type, event timestamp, relevant issue, actor when available, previous value, new value, and field-level attributes. The extractor restricts events to the selected instance window and deduplicates exact status transitions appearing in both status history and the full changelog. This avoids double-counting a workflow change simply because Jira exposes it in two structures.

Jira projects customize their fields extensively. A global audit identified 184 distinct changelog field types across the selected instances. Known variants were mapped to standardised event types, including STATUS_CHANGE, SPRINT_CHANGE, ASSIGNEE_CHANGE, PRIORITY_CHANGE and VERSION_CHANGE. Repository-specific fields were retained using the CUSTOM_FIELD_CHANGE category, which preserves the original Jira field name and its previous and updated values. The final corpus contains 14,138 such events and zero residual OTHER_CHANGE events.

Table 5.2 Information Extraction Methods for Different Project Data Sources

Evidence	Extraction approach	Reason
Issue key, project, assignee, component	Structured-field parsing	Explicit fields should remain authoritative
Requirement / Jira references	Regular expressions	Identifiers have recognizable syntax
Status, sprint, priority and other changes	Changelog mapping + timestamp filtering	Preserves old/new state and chronology
System, feature, team, component mentions	GLiNER semantic extraction	Concepts occur in free-form language
Actions, decisions and blocker statements	Rule-based classification	Retains assertion state and context

Example of the standardized representation

{

"source": {"source_type": "JIRA", "source_id": "DOCS-9947"},
"entities": [{"type": "ISSUE", "name": "DOCS-9947"}], "events": [{
"event_type": "STATUS_CHANGE",
"event_time": "<timestamp from Jira history>", "from_value": "Open", "to_value": "In Progress" }],
"provenance": {"source_type": "JIRA", "file_name": "<source file>"}
}

Table 5.3 Summary of Extracted Entities, Relationships and Events

Output category	Count
Total source records	5,827
Entity occurrences (not yet deduplicated)	61,392
Relationship occurrences	63,305
Timestamped events	82,577
Dependency evidence records	7,709
Blocker evidence records	443
Action evidence records	207
Decision evidence records	414

5.3 Entity Resolution

Entity resolution reconciles source-scoped mentions into canonical entities within each project instance. The process produces a portable canonical dataset before database-specific import,

preserving the distinction between mention-level evidence and consolidated graph identities. Canonicalisation prevents the same issue, requirement or project actor from being treated as unrelated nodes merely because it appears in different source records. The exported node and edge files also support reproducible construction and integrity checking.

5.4 Temporal Knowledge Graph

The graph construction process uses the frozen entity-resolution exports and original source records to build stable nodes and relationships. The representation includes issue and other project entities, event nodes, typed links and evidence references. Its time model distinguishes event_time (when an event occurred), valid_from/valid_to (the interval during which a fact applies) and observed_at (when the information became available). These attributes are not interchangeable: an event can describe an earlier state yet become visible only after a later update.

The graph is exported in portable formats together with Neo4j-ready CSV files, constraints, import scripts and validation diagnostics. Graph creation can therefore be inspected independently of an active Neo4j database. The feature-engineering implementation reports 96,487 temporal graph nodes and 164,101 edges in the loaded experimental export; these counts describe that specific processed dataset, not a production graph.

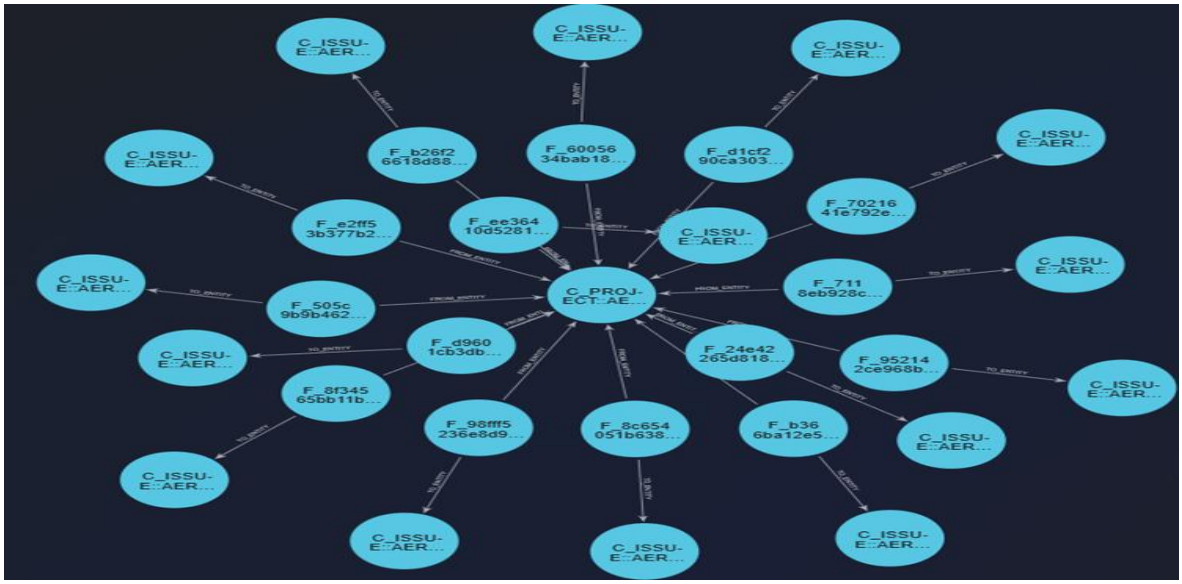


Figure 5.2 Sample Temporal Knowledge Graph for Project AEROGEAR-I01, Showing Project–Issue Relationships and Cross-Issue Associations.

5.5 Knowledge Quality and Traceability

The system validates the Temporal Knowledge Graph by checking entity identifiers, relationships, timestamps, and source references to ensure consistency and traceability. A source-to-graph audit tracks Jira records through information extraction, entity resolution, and graph construction to verify how the original project information is represented. In the sampled evaluation, 35 out of 60 selected Jira issues were identified during the initial extraction stage, helping assess information coverage across the pipeline.

The system also evaluates the accuracy of temporal events and their chronological relationships by comparing extracted graph information with reference records. Precision and recall are calculated for comparable event categories, while temporal ordering is assessed using matched event pairs. These evaluations help measure the quality of knowledge representation and the system's ability to preserve project history for subsequent risk prediction and investigation.

6. RISK PREDICTION AND REASONING

6.1 Weekly Snapshots and Outcome Labels

Prediction examples are reconstructed at project-week checkpoints. The ground-truth generation process reconstructs historical Jira status, dependency intervals and graph visibility, and produces 987 eligible snapshots across 30 projects. For each checkpoint, a risk label indicates whether a qualifying future event occurs within the subsequent 14 days. Future observations are used to establish the supervised target, not as predictive features at the checkpoint.

Ground-truth labels are generated by examining qualifying project events within the 14 days following each checkpoint. Schedule Risk is labelled when at least three distinct issues experience eligible workflow disruptions, such as reopening or status regression. Scope Risk is labelled when at least three distinct issues are affected by structural scope changes, including epic-child additions, epic reassignment or issue reclassification. Dependency Risk is labelled when at least three distinct downstream issues are affected by eligible newly added blocking dependencies. The scope-event preparation also excludes 29 suspected administrative reclassifications, retaining 938 eligible structural scope events. Validation checks verify label completeness, temporal consistency and project-week uniqueness.

6.2 Temporal and Graph Feature Engineering

Each historical snapshot is transformed into a fixed numerical representation. The feature-engineering pipeline combines Jira workflow state, rolling event activity, dependency-network structure, temporal graph connectivity and historical scope-change activity. Examples include counts of open and reopened issues, recent event volumes, event growth, active dependency links, blocked and blocking issue counts, graph nodes and edges visible at the checkpoint, and scope changes in preceding windows.

Following feedback on the initial XGBoost-based approach, we experimented with alternative tabular learning models, including TabPFN 2.5 and CatBoost. Based on the experimental results, TabPFN 2.5 was selected for Schedule Risk, while CatBoost was selected for Scope and Dependency Risks. The models use the same 34 selected numerical features, with their comparative performance discussed in Section 8.

The pipeline also screens concentrated bulk updates and stabilises event-growth measurements so that administrative bursts or small denominators do not dominate the signal. From a larger engineered feature table, 34 numerical predictors are selected for modelling. The final matrix contains 987 checkpoint records, 34 predictors and three binary risk labels, with validation checks for missing values and consistency with consolidated ground truth.

Table 6.1 Selected Numerical Features for Project Risk Prediction

Feature category	Selected features	Count
Historical Jira Workflow	total_issues, open_issue_ratio, reopened_issues, blocked_status_issues, unknown_status_ratio, recent_status_changes_14d_bulk_adjusted, status_change_ratio_14d_bulk_adjusted	7
Temporal Event Activity	log_events_last_7d, log_events_last_28d, event_log_growth_bulk_adjusted, event_volatility_bulk_adjusted, active_event_days_14d_bulk_adjusted, new_issues_last_14d, project_age_days	7
Dependency Graph	dependency_links, blocked_issue_count, max_blocking_degree, max_blocker_count, dependency_components, max_dependency_chain, dependency_graph_density, dependency_node_coverage	8
Structural Temporal KG	kg_entity_nodes, kg_active_entities_14d, kg_max_entity_degree, kg_entity_concentration, kg_new_entities_14d	5
Historical Scope Changes	scope_changes_last_14d, scope_changes_last_28d, epic_child_additions_14d, epic_reassignments_14d, issue_reclassifications_14d, scope_affected_issues_14d, scope_change_log_growth	7
Total		34

Table 6.1 summarises the selected features and their respective categories. Each category represents a different aspect of project activity, allowing the system to consider both individual task behaviour and relationships across the project. Together, these features provide a structured view of project conditions that supports the identification of emerging risk patterns.

6.3 Final Predictive Models

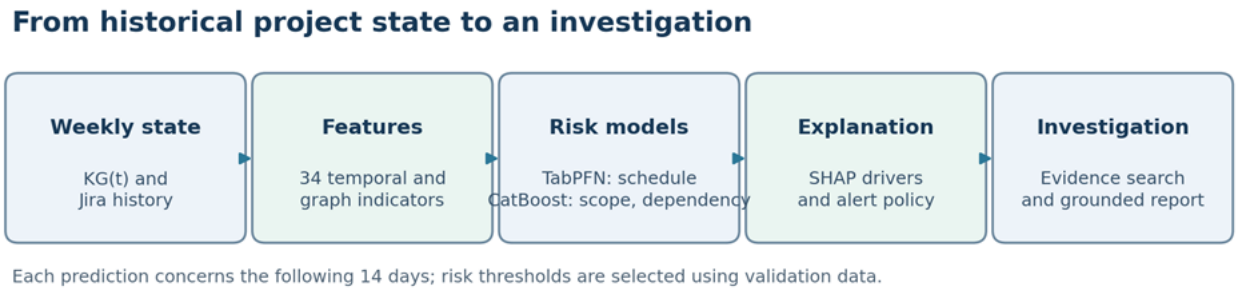


Figure 6.1 Weekly prediction and risk-triggered investigation workflow

Table 6.2 Predictive Models for Schedule, Scope and Dependency Risks

Risk target	Model	Interpretation
Schedule Risk	TabPFN 2.5	Pretrained tabular classifier applied to the selected numerical feature representation
Scope Risk	CatBoost	Gradient-boosted decision-tree classifier
Dependency Risk	CatBoost	Gradient-boosted decision-tree classifier

The final predictive system uses TabPFN 2.5 for Schedule Risk and CatBoost classifiers for Scope and Dependency Risks. Each model generates a risk score for its corresponding category, which is compared against a validation-selected threshold to identify potential risks. The CatBoost models are configured with 250 iterations, a tree depth of 3, a learning rate of 0.035 and L2 regularisation of 8. TabPFN 2.5 uses four estimators for Schedule Risk prediction. This combination allows the system to apply different tabular learning approaches to the three risk categories, with their predictive performance evaluated separately.

6.4 Model Validation and Temporal Generalisation

The main model workflow uses project-grouped training and validation logic to reduce mixing of related project checkpoints. Decision thresholds are selected on validation data rather than adjusted against the test outcomes. A separate chronological experiment tests within-project generalisation by fitting on earlier history, using later validation weeks and evaluating still-later weeks. Its configured windows begin with Weeks 1–18 for historical fitting, Weeks 21–24 for validation and Week 27 onward for future testing, with intervening gaps to protect the 14-day outcome horizon.

This distinction is important: generalisation to unseen projects and forecasting future weeks of an already observed project are different evaluation questions. Their results should not be pooled or interpreted as a single deployment accuracy estimate.

6.5 SHAP Explanations and Investigation Trigger

SHAP is used to attribute a selected prediction to its numerical features. The system uses a permutation-based SHAP approach for TabPFN and model-compatible explanations for CatBoost, generating the most influential drivers on demand. Feature attribution answers which model inputs influenced a risk score; it does not, by itself, prove that those features caused the underlying project problem.

A stateful investigation policy combines probability thresholds, risk context and alert history to decide when an investigation should be initiated. The system supports chronological replay and persistent alert scheduling, and produces a structured handoff containing project instance, checkpoint, risk type, score and selected explanatory drivers. This avoids requiring a full graph investigation for every model inference.

6.6 Agentic GraphRAG Investigation

When a risk alert is selected, the investigation component analyses the available project evidence and generates a report linking its findings to the relevant source records. The system uses

Qwen2.5-3B-Instruct, a lightweight language model, to analyse project events and generate evidence-based explanations. The investigation process takes structured inputs containing the project instance, checkpoint, risk category, and prediction details to identify relevant events and explain the factors associated with the detected risk.

Before generation, the system checks whether each evidence record belongs to the correct project instance, has usable provenance and was available at the historical checkpoint. Unsafe records are excluded. The investigation then ranks the remaining evidence and may select additional records from the existing historical retrieval cache for the same project instance. This follow-up step does not issue a fresh graph database query.

The model receives a bounded set of admissible evidence and is instructed to produce structured observations with supporting evidence identifiers. The system parses the response into a report and checks its format, citations and historical eligibility. Where evidence is insufficient, the report may be marked inconclusive instead of presenting an unsupported root cause.

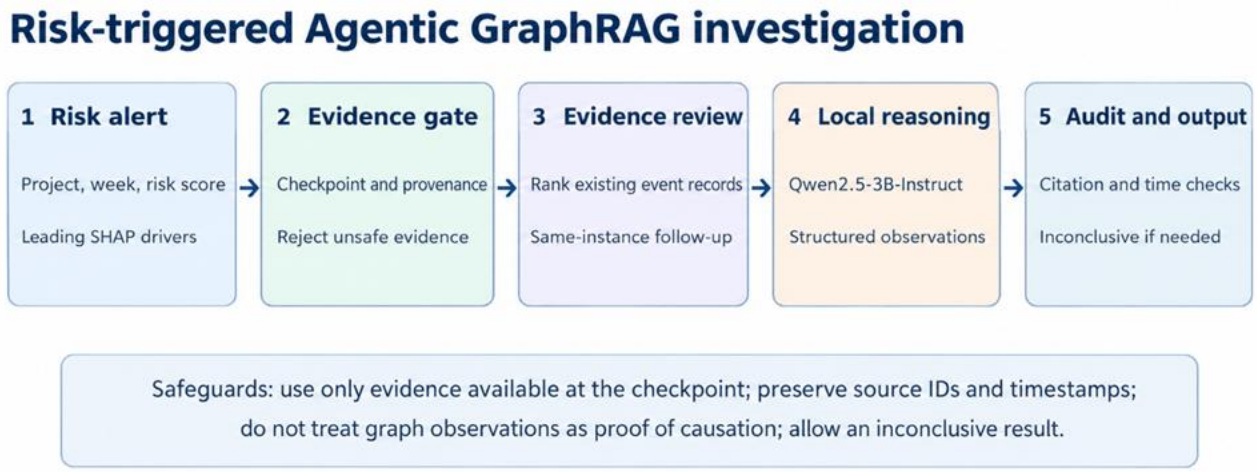


Figure 6.2 Risk-triggered investigation and evidence checks

The completed full-queue run generated 139 reports for 139 unique investigation requests (74 validation requests and 65 exploratory test requests). Of these, 121 reported limited evidence observations and 18 were inconclusive. The workflow identified 78 unsafe evidence records among 1,270 retrieved records and found no records classified as verified original source text. These figures describe execution and automated checks, not independently proven explanation accuracy.

The system processed 139 investigation requests, comprising 74 validation cases and 65 exploratory test cases. Of the generated reports, 137 were produced using Qwen2.5-3B-Instruct, while two followed rule-based abstention due to insufficient evidence. Eight reports were flagged for further review. These results demonstrate the system's ability to process risk alerts and generate structured investigation reports, with additional verification used to assess the reliability of the findings.

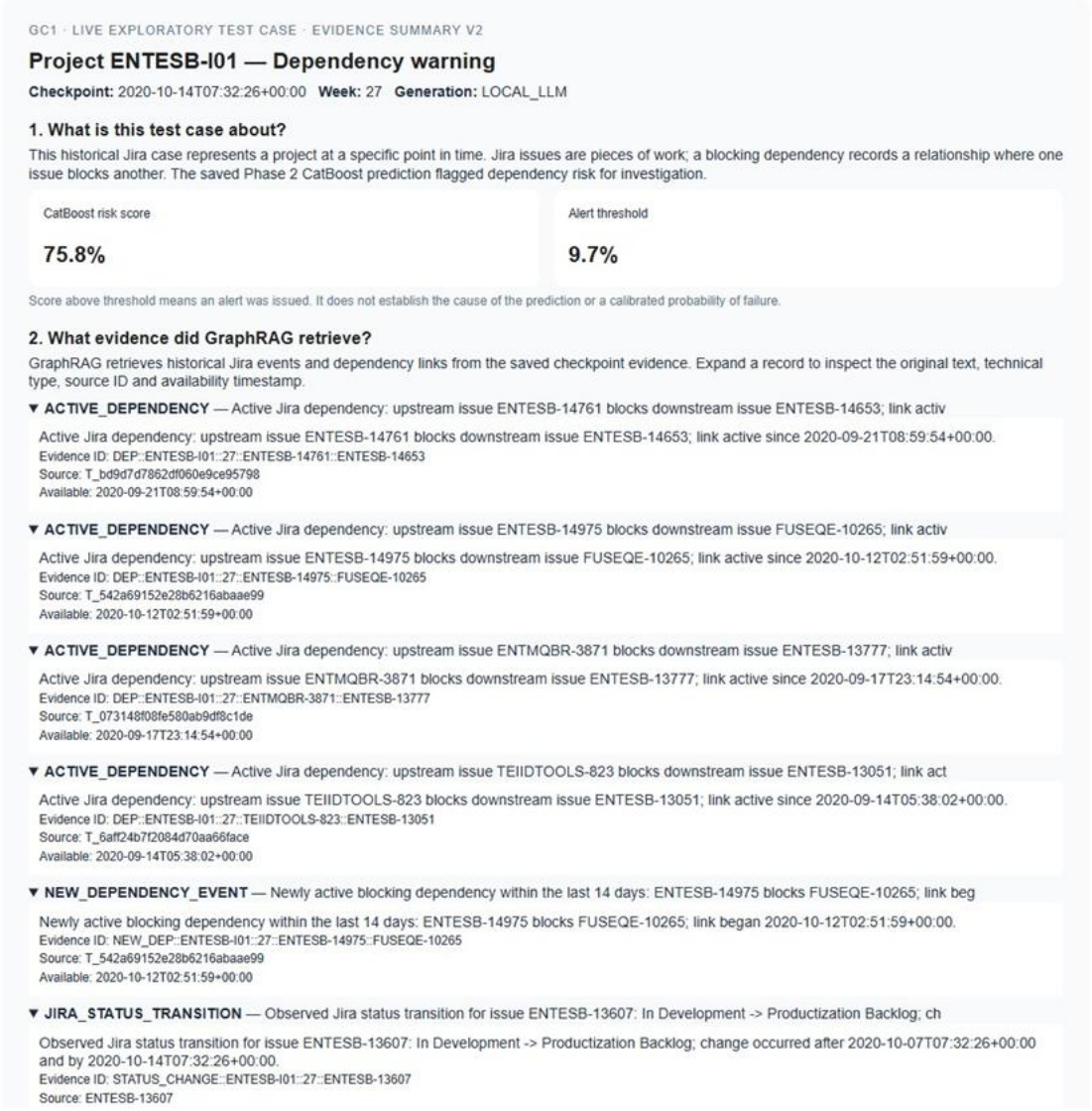
6.6.1 Example: Dependency Risk Investigation

The system was tested on project ENTESB-I01 at Week 27 (checkpoint: 14 October 2020). The saved CatBoost Dependency Risk score was 75.8%, above the configured alert threshold of 9.7%.

This triggered an investigation. The score indicates that the alert rule was met; it is not a calibrated probability of project failure.

The historical evidence included four active blocking dependencies, one newly active dependency within the previous 14 days, and a Jira status-transition record. Qwen2.5-3B-Instruct summarised two findings: several blocking dependencies were active, and a new dependency connected ENTESB-14975 with FUSEQE-10265. The report did not claim that these dependencies had caused delays or scope changes.

The source audit resolved all five cited evidence occurrences and confirmed historical availability for the supplied records. This verifies citation resolution and checkpoint eligibility for this case, not the factual accuracy of every possible interpretation.



GC1 · LIVE EXPLORATORY TEST CASE · EVIDENCE SUMMARY V2

Project ENTESB-I01 — Dependency warning

Checkpoint: 2020-10-14T07:32:26+00:00 **Week:** 27 **Generation:** LOCAL_LLM

1. What is this test case about?

This historical Jira case represents a project at a specific point in time. Jira issues are pieces of work; a blocking dependency records a relationship where one issue blocks another. The saved Phase 2 CatBoost prediction flagged dependency risk for investigation.

CatBoost risk score	Alert threshold
75.8%	9.7%

Score above threshold means an alert was issued. It does not establish the cause of the prediction or a calibrated probability of failure.

2. What evidence did GraphRAG retrieve?

GraphRAG retrieves historical Jira events and dependency links from the saved checkpoint evidence. Expand a record to inspect the original text, technical type, source ID and availability timestamp.

▼ **ACTIVE_DEPENDENCY** — Active Jira dependency: upstream issue ENTESB-14761 blocks downstream issue ENTESB-14653; link activ

Active Jira dependency: upstream issue ENTESB-14761 blocks downstream issue ENTESB-14653; link active since 2020-09-21T08:59:54+00:00.
Evidence ID: DEP::ENTESB-I01::27::ENTESB-14761::ENTESB-14653
Source: T_bd9d7d7862df060e9ce95798
Available: 2020-09-21T08:59:54+00:00

▼ **ACTIVE_DEPENDENCY** — Active Jira dependency: upstream issue ENTESB-14975 blocks downstream issue FUSEQE-10265; link activ

Active Jira dependency: upstream issue ENTESB-14975 blocks downstream issue FUSEQE-10265; link active since 2020-10-12T02:51:59+00:00.
Evidence ID: DEP::ENTESB-I01::27::ENTESB-14975::FUSEQE-10265
Source: T_542a69152e28b6216abaae99
Available: 2020-10-12T02:51:59+00:00

▼ **ACTIVE_DEPENDENCY** — Active Jira dependency: upstream issue ENTMQBR-3871 blocks downstream issue ENTESB-13777; link activ

Active Jira dependency: upstream issue ENTMQBR-3871 blocks downstream issue ENTESB-13777; link active since 2020-09-17T23:14:54+00:00.
Evidence ID: DEP::ENTESB-I01::27::ENTMQBR-3871::ENTESB-13777
Source: T_073148f08fe580ab9df8c1de
Available: 2020-09-17T23:14:54+00:00

▼ **ACTIVE_DEPENDENCY** — Active Jira dependency: upstream issue TEIIDTOOLS-823 blocks downstream issue ENTESB-13051; link act

Active Jira dependency: upstream issue TEIIDTOOLS-823 blocks downstream issue ENTESB-13051; link active since 2020-09-14T05:38:02+00:00.
Evidence ID: DEP::ENTESB-I01::27::TEIIDTOOLS-823::ENTESB-13051
Source: T_6aff24b7f2084d70aa66face
Available: 2020-09-14T05:38:02+00:00

▼ **NEW_DEPENDENCY_EVENT** — Newly active blocking dependency within the last 14 days: ENTESB-14975 blocks FUSEQE-10265; link beg

Newly active blocking dependency within the last 14 days: ENTESB-14975 blocks FUSEQE-10265; link began 2020-10-12T02:51:59+00:00.
Evidence ID: NEW_DEP::ENTESB-I01::27::ENTESB-14975::FUSEQE-10265
Source: T_542a69152e28b6216abaae99
Available: 2020-10-12T02:51:59+00:00

▼ **JIRA_STATUS_TRANSITION** — Observed Jira status transition for issue ENTESB-13607: In Development -> Productization Backlog; ch

Observed Jira status transition for issue ENTESB-13607: In Development -> Productization Backlog; change occurred after 2020-10-07T07:32:26+00:00 and by 2020-10-14T07:32:26+00:00.
Evidence ID: STATUS_CHANGE::ENTESB-I01::27::ENTESB-13607
Source: ENTESB-13607

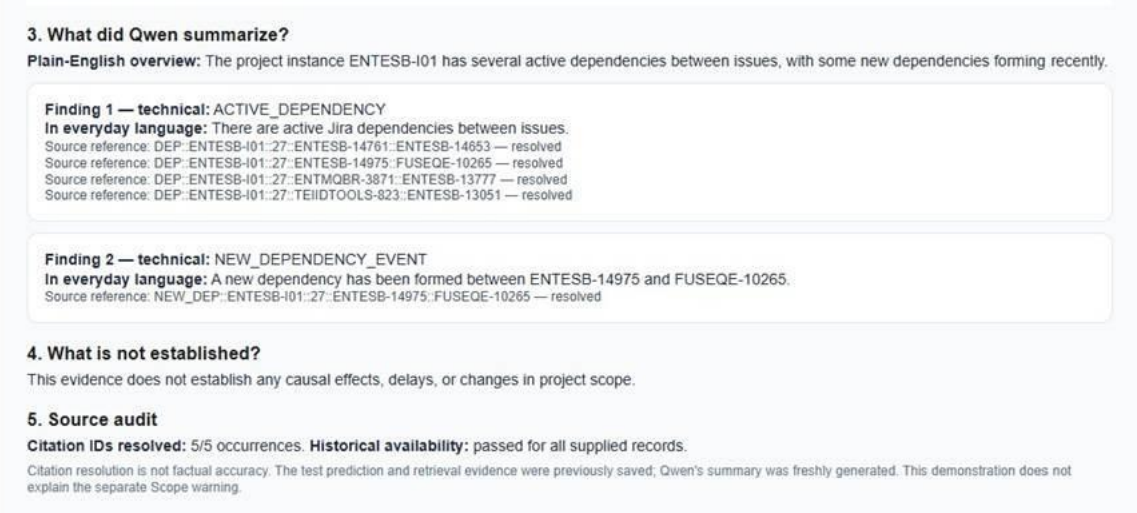


Figure 6.3. Agentic GraphRAG Investigation Output for Project ENTESB-I01: Retrieved Evidence, Qwen Findings and Source Verification

7. SYSTEM DEVELOPMENT AND IMPLEMENTATION

The system was developed as a modular pipeline that integrates knowledge extraction, temporal graph construction, machine learning, and evidence-based investigation. Each component was implemented and validated independently before being connected to the complete workflow.

The implementation processes project information from 30 enterprise project instances, combining public Jira histories with generated requirement documents and meeting records. The completed extraction contains 5,827 source records, from which entities, relationships, and timestamped events are identified. The predictive pipeline reconstructs 987 weekly project checkpoints and generates 34 numerical features for risk prediction.

7.1 System Development Approach

The system follows a modular development approach, where each stage produces structured outputs that serve as inputs to subsequent components. This allows individual modules to be developed, tested, and updated without modifying the entire system.

Python is used for data processing, feature engineering, machine learning, and investigation workflows. The Temporal Knowledge Graph is constructed using resolved project entities and relationships, with Neo4j-compatible graph exports supporting graph storage and querying. The predictive models and investigation components operate on structured intermediate outputs, ensuring consistent data exchange between modules.

The implementation also preserves project identifiers, timestamps, and source references throughout the pipeline. These details allow project events and generated explanations to be traced back to their original records.

7.2 Implementation of System Components

The system consists of several components, each responsible for a specific part of the processing workflow. Table 7.1 summarises their implementation and outputs.

Table 7.1: System Implementation Components and Their Outputs

Component	Implementation	Output
Data Preparation	Python, pandas, MongoDB and public Jira data processing	Standardised Jira, PRD and meeting records
Information Extraction	Structured parsing, regular expressions and GLiNER	Entities, relationships and timestamped events in JSONL
Entity Resolution	Entity matching and canonicalisation across project sources	Resolved entities and relationships
Temporal Knowledge Graph	Temporal graph construction and Neo4j-compatible exports	Timestamped graph nodes, relationships and evidence references
Feature Engineering	Temporal and graph-based feature calculations	34 numerical features for each project-week checkpoint
Risk Prediction	TabPFN 2.5 for Schedule Risk; CatBoost for Scope and Dependency Risks	Risk scores and classifications
Prediction Explanation	SHAP feature attribution	Feature contributions for selected predictions
Investigation Scheduling	Risk thresholds and historical alert states	Structured investigation requests
Evidence Retrieval	Temporal graph traversal, FAISS, BM25, Reciprocal Rank Fusion and cross-encoder reranking	Ranked historical evidence
Agentic GraphRAG	Qwen2.5-3B-Instruct, evidence filtering and citation verification	Structured investigation reports

The components exchange information through structured datasets, graph exports, prediction records, and investigation requests. This design allows the system to retain historical context from the original project data through to the final investigation report.

7.3 System Integration and Execution

The integration connects the knowledge representation, predictive modelling, and investigation components into a single processing workflow. The process begins with the preparation of Jira records and supporting project documents. Extracted entities and relationships are resolved

and used to construct the Temporal Knowledge Graph. At each weekly checkpoint, the system reconstructs the available project state and calculates the corresponding numerical features.

The feature vectors are passed to the trained TabPFN and CatBoost models to estimate Schedule, Scope, and Dependency Risks over the following 14 days. Risk-specific thresholds are applied to identify alerts, while SHAP provides feature-level explanations for selected predictions.

When an alert requires investigation, the system retrieves relevant historical project evidence using graph-based and hybrid retrieval techniques. The retrieved records are filtered according to project identity, source availability, and checkpoint time before being passed to the Agentic GraphRAG component.

The investigation component generates a structured report containing the risk context, supporting evidence, observations, citations, and conclusion status. Each stage stores its outputs separately, allowing the system to reproduce previous results and rerun individual components when necessary.

7.4 Agentic GraphRAG Implementation

7.4.1 Investigation Execution and Report Generation

The investigation component uses Qwen2.5-3B-Instruct, a lightweight language model, to analyse the evidence associated with selected risk alerts. The model is loaded through the Transformers library and receives structured investigation requests containing the project identifier, checkpoint, risk category, prediction details, and retrieved evidence.

Before generation, the system checks the historical eligibility and source references of the retrieved records. The generation context is limited to 12 evidence items, with an additional follow-up mechanism that can select up to four relevant records from the existing historical retrieval queue for the same project instance.

The model generates a structured JSON report containing a summary, evidence-based observations, citations, and a conclusion status. The output is then processed into a readable investigation report with five main parts:

1. Project context and predicted risk
2. Retrieved evidence with source identifiers and timestamps
3. Qwen-generated findings and explanations
4. Findings that cannot be established from the available evidence
5. Source verification results

Each investigation is saved separately, allowing completed reports to be retained and interrupted processing to resume without repeating previously completed requests.

The interactive evidence-summary interface also allows users to browse the 139 saved V5 reports and inspect risk warnings, historical event types, source identifiers and Qwen findings. A separate exploratory demonstration can generate a fresh evidence summary for a selected case; this output is distinguished from the frozen V5 evaluation run.

7.4.2 Evidence and Citation Verification

The system validates generated investigation reports by checking their structure, citation references, and historical evidence availability. Source identifiers are used to connect the generated observations to the retrieved project records.

A separate source-level verification process compares generated claims with dated records from the original Jira dataset. This helps identify unsupported statements, incorrect citations, and inconsistencies between the generated explanation and the available project evidence.

For example, in the ENTESB-I01 Dependency Risk investigation, all five cited evidence occurrences were successfully resolved, and the supporting records passed the historical availability checks. This demonstrates the system's ability to maintain traceable references between investigation findings and source records.

The broader verification process also evaluates whether the retrieved evidence directly supports the generated claims, distinguishing citation validity from factual correctness.

7.4.3 Faithfulness Refinement

An additional refinement process was implemented to improve the reliability of generated explanations. This process screens reports for citation problems and unsupported inferences and allows the language model to revise its response when necessary.

The refinement was tested on 30 investigation cases. Nine reports contained limited evidence-based observations, while 21 were classified as inconclusive. These results reflect a more cautious generation process that prioritises supported findings and allows the system to indicate when the available evidence is insufficient.

The investigation also distinguishes between evidence retrieval and explanation generation. Retrieved records must contain information relevant to the predicted risk before they can support a meaningful explanation. For example, a Scope Risk investigation requires historical structural changes, such as epic reassignment or changes in issue classification, rather than relying only on general Jira status records.

This approach helps the system produce explanations based on relevant historical evidence while reducing unsupported conclusions.

7.5 System Validation and Integrity Checks

Validation checks are performed throughout the implementation to ensure that data remains consistent as it moves between components.

Table 7.2: Validation Checks Across the System

Component	Validation performed
Data Preparation	Record completeness, project identifiers, timestamps and source consistency

Component	Validation performed
Information Extraction	Required schema fields, event types and provenance
Entity Resolution	Canonical identifiers and relationship consistency
Temporal Knowledge Graph	Node uniqueness, relationship endpoints and temporal validity
Feature Engineering	Missing values, project-week uniqueness and feature consistency
Risk Prediction	Project-grouped data partitions, validation-based thresholds and chronological evaluation
Agentic GraphRAG	Evidence eligibility, report structure, citation resolution and source-level verification

The completed extraction audit covered all 30 project instances and validated 5,827 source records. The extracted information included 61,392 entity occurrences, 63,305 relationship occurrences, and 82,577 timestamped events before entity resolution and deduplication.

The integrated investigation pipeline was validated through batch execution, structured report checks, citation resolution and source-level verification. These checks assess the completeness of generated reports, the traceability of supporting evidence and the consistency of investigation outputs. Detailed execution statistics and verification results are presented in Section 8.6.

The source-level verification examined 155 generated claims. Of these, 146 had related source facts but were not fully verified, while nine remained unresolved. Eight claims contained citation errors. These checks help identify areas where evidence retrieval and explanation generation require further refinement.

7.6 Engineering Considerations and Limitations

The system has been implemented as a research prototype with independently developed and integrated components. Its modular design supports reproducible execution and allows individual stages to be evaluated and improved.

Several considerations remain important when interpreting the results. The experimental dataset combines public Jira histories with generated project documents, and the quality of extracted knowledge depends on the completeness of the original records. Differences in timestamp availability, event coverage, and project activity can affect temporal graph construction and subsequent predictions.

The investigation component uses previously retrieved historical evidence for follow-up selection. It also applies citation and source checks to support traceability, although a valid citation does not necessarily establish that every generated statement is factually correct.

The implemented workflow demonstrates the integration of temporal knowledge representation, supervised risk prediction, and evidence-based investigation. Further evaluation of

predictive performance, evidence relevance, and explanation accuracy is presented in the Experiments and Findings sections.

8. EXPERIMENTS AND EVALUATION

The experiments examined the complete reasoning pipeline, from risk modelling to evidence retrieval. For prediction, we compared Logistic Regression, XGBoost, TabPFN 2.5 and CatBoost using project-grouped validation and metrics including PR-AUC, ROC-AUC and F1-score. We also tested additional temporal-change and project-specific features, alongside general and event-focused hybrid retrieval. These experiments informed the final selection of TabPFN for Schedule Risk, CatBoost for Scope and Dependency Risks, and event-focused retrieval for investigation.

Evaluation then considers the three implemented phases in sequence: information extraction and temporal knowledge representation (Phase 1), supervised risk prediction (Phase 2), and GraphRAG retrieval with Qwen-based evidence summarisation (Phase 3). Each phase is assessed against its own reference data and metrics, so results are reported separately rather than combined into a single score.

8.1 Evaluation

8.1.1 Evaluation Methodology

Phase 1 uses precision, recall and F1 to compare extracted mentions, relationships and graph events with reference annotations. Entity-resolution scoring uses manually assigned judgments for assessable pairs. Phase 2 uses F1, precision–recall area under the curve (PR-AUC) and area under the receiver operating characteristic curve (AUROC), distinguishing validation from exploratory chronological testing. Phase 3 evaluates retrieval relevance using Precision@5 and examines generated claims through AI-assisted evidence-support judgments, citation audits and source-level checks.

F1 is the harmonic mean of precision and recall. Precision@5 measures the proportion of the five highest-ranked retrieved items judged relevant. AUROC assesses ranking discrimination across decision thresholds; PR-AUC focuses on positive-case retrieval in imbalanced targets. A successful execution, a resolved citation and a supported claim are distinct outcomes.

8.1.2 Benchmark Selection and Reference Annotation

The Phase 1 Information Extraction pipeline was evaluated using a frozen benchmark comprising 30 passages from 18 distinct source documents across six selected projects: JBIDE, SAK, MNG, MC, HBASE and SERVER. Each project contributed five passages, consisting of two Product Requirements Document (PRD) passages and three meeting-record passages. This provided consistent project-level coverage while incorporating both requirements-oriented and discussion-oriented source material.

The benchmark was manually annotated to establish reference entities and relationships. The final ground-truth files contained 300 entity annotations across ten entity categories and 219 relationship

annotations across eight relationship categories. Entity annotations recorded the original mention, entity type and canonical name, while relationship annotations identified the subject, relationship type and object. Each annotation was linked to its corresponding benchmark passage through a unique sample identifier. This enabled the extraction outputs to be compared against the reference annotations at the passage level.

The evaluation assessed entity mention detection, entity type classification and relationship extraction using Precision, Recall and F1-score. The benchmark was frozen before final scoring to maintain a consistent evaluation set across experiments.

Although the benchmark provided balanced coverage across the six selected projects, it was relatively small and was not established as a statistically representative sample of the complete dataset. Consequently, the reported results describe extraction performance on the selected passages rather than general performance across all project documents.

Table 8.1: Component and Benchmark specification table

Component	Benchmark
Project instances	JBIDE, SAK, MNG, MC, HBASE, SERVER
Total passages	30
Document sources	PRD and meeting records
Reference standard	Human entity and relationship annotations
Evaluation approach	Document- and passage-aligned matching
Metrics	Precision, Recall, F1-score

8.2 Phase 1 — Information Extraction and Entity Resolution

The information-extraction evaluation examined the recovery and classification of entity mentions and the identification of relationships. Results indicate moderate agreement with the annotated reference set; assigning entity types was the weakest of the three extraction tasks.

Table 8.2 Evaluation Results for Entity Extraction, Relationship Extraction and Entity Resolution

Evaluation task	F1-score	Interpretation
Entity mention detection	55.9%	Moderate recovery of expected mentions
Entity type classification	49.5%	Difficulty assigning the correct semantic category
Relationship extraction	56.7%	Partial recovery of linked project facts
Entity resolution	100.0%	Agreement on 103 manually scored pairs; 41 uncertain pairs excluded

The entity-resolution result must be read with caution. A reviewer manually labelled 103 assessable candidate pairs, while 41 were marked uncertain because project-specific terminology or original Jira context could not be confidently resolved. The 100% F1 therefore measures agreement with those manual judgments on the included subset; it does not demonstrate error-free resolution across all project entities or independent source verification.

8.3 Phase 1 — Temporal Knowledge Graph Fidelity

The temporal graph audit compared graph-materialised events against curated Jira event references. Exact matching is the primary measure because event identity and chronology are important for historical risk analysis. A one-second tolerance was examined as a diagnostic, not as a replacement for exact matching.

Table 8.3 Temporal Knowledge Graph Fidelity Evaluation Results

Evaluation task	F1-score	Interpretation
Exact temporal event match	44.5%	Primary strict matching result
Event match with ±1-second tolerance	75.8%	Diagnostic sensitivity to small timestamp offsets
Extraction-to-graph retention	42.3%	Exact information retention through graph construction

Allowing a one-second difference recovered 8,117 additional near matches. In a 40-pair follow-up, 30 had confirmed details, eight matched only on fields and two remained ambiguous. This supports reporting the exact-match F1 of 44.5% as the primary result. The audit additionally identified 35 reversed time intervals and 68 observation-time anomalies. Structural connectivity checks should not be interpreted as evidence that each graph fact is correct.

8.4 Phase 2 — Risk Prediction

The final prediction configuration uses TabPFN 2.5 for Schedule risk and CatBoost for Scope and Dependency risk. Labels refer to qualifying events in the 14 days following each historical checkpoint. Thresholds were selected using validation data. The later chronological test is described as exploratory because it had been examined during development and is not an untouched external holdout.

Table 8.4 Risk Prediction Performance on Validation and Exploratory Test Sets

Evaluation metric	Validation	Exploratory test
Overall F1 across risk targets	37.2%	44.1%
Schedule risk F1	46.5%	62.6%
Scope risk F1	44.9%	45.0%
Dependency risk F1	20.0%	24.7%
Mean PR-AUC	28.7%	31.5%
Mean AUROC	65.3%	76.1%

Schedule risk achieved the strongest exploratory F1, while Dependency risk remained the weakest target, consistent with false-positive alerts. The exploratory AUROC suggests some

discrimination between higher- and lower-risk cases, but does not establish reliable deployment performance or probability calibration. Scope performance remained modest. The validation and exploratory results should be shown separately rather than combined into a single accuracy score.

The label-generation checks passed 30 of 30 automated consistency tests; dependency-label reproduction matched 816 of 816 eligible events and 987 of 987 labels. These checks demonstrate reproducibility of the implemented label calculations, not independent verification of every source event. A complete guarantee against future-data leakage and exact lead time before a failure were not established.

8.5 Phase 3 — GraphRAG Retrieval Relevance

The retrieval experiment assessed whether the highest-ranked historical evidence items directly addressed the selected risk warning. Relevance judgments were AI-assisted, and the strict screening rule penalised general project activity that did not specifically support the risk category.

Table 8.5 GraphRAG Evidence Retrieval Relevance Evaluation

Retrieval metric	Precision@5	Assessment basis
Overall	37.3%	Original AI-assisted relevance evaluation
Dependency	61.8%	Original risk-specific evaluation
Schedule	36.7%	Original risk-specific evaluation

Dependency Risk achieved the highest retrieval relevance in this evaluation. Scope Risk retrieval showed lower relevance, as historically valid Jira records, such as status transitions, did not always provide direct evidence of structural scope changes. These results measure the relevance of retrieved evidence rather than the quality of Qwen-generated explanations and require further human validation.

8.6 Phase 3 — Qwen Report and Evidence-Support Evaluation

The frozen V5 run processed all 139 investigation requests: 74 validation and 65 exploratory-test requests. Of the outputs, 137 used local Qwen generation and two used rule-based abstention; 121 contained limited evidence observations and 18 were inconclusive. These are execution and report-category outcomes, not measures of factual accuracy.

Table 8.6 Qwen Investigation Report and Evidence-Support Evaluation Results

Evaluation check	Result	Interpretation
AI-assisted claim support (40 claims)	70.0% fully supported	28/40 judged fully supported
Unsupported claims (40 claims)	12.5%	5/40 unsupported; seven partly supported
AI-assisted review (155 claims)	69.0% fully supported	11.6% judged unsupported
Initial citation flags	8 of 139 reports	Reports requiring reference review
Source-level comparison (155 claims)	146 related but not fully verified; 9 unresolved	No established overall original-source factual accuracy

The 40-claim and 155-claim AI-assisted assessments are preliminary and must not be described as independent human-verified accuracy. The source-level comparison located related facts for many claims but did not establish exact factual entailment. Citation resolution checks whether a referenced evidence identifier exists; it cannot independently confirm the correctness of the model's interpretation.

In the ENTESB-I01 live exploratory-test example, five of five cited evidence occurrences resolved and all supplied records passed checkpoint-availability checks. This is a case-level traceability demonstration, not a system-wide citation accuracy rate. The interactive evidence-summary prompt used for that single-case demonstration is distinct from the frozen V5 batch-generation prompt.

8.7 Additional Retrieval and Faithfulness Experiments

An additional retrieval experiment evaluated the effect of risk-aware reranking on 30 investigation cases comprising 173 unique evidence items. Both the baseline and risk-aware approaches achieved a Precision@5 of 49.3%, indicating no overall improvement from reranking in this experiment. Risk-specific results were 94% for Dependency, 54% for Schedule and 0% for Scope. This experiment used a separate evaluation sample and protocol from the initial retrieval assessment in Section 8.5, which reported an overall Precision@5 of 37.3%. The two evaluations therefore provide complementary findings on retrieval performance under different experimental settings.

A further 30-case faithfulness-focused refinement produced nine reports with limited evidence observations and 21 inconclusive reports. The more cautious outcome pattern may indicate increased abstention, but it is not a validated semantic-faithfulness improvement. Both experiments are developmental comparisons; the final evaluated implementation remains V5.

8.8 Consolidated Interpretation and Evaluation Boundaries

Across phases, the evaluation demonstrates a functioning, inspectable research prototype rather than production-level predictive or explanatory accuracy. Phase 1 shows substantial gaps in exact temporal event matching; Phase 2 shows uneven predictive performance, particularly for Dependency risk; and Phase 3 shows that historical evidence can be retrieved and summarised but that strict relevance and original-source factual verification remain challenging.

The system's final Qwen role is best described as evidence-grounded summarisation associated with risk warnings. It does not establish why a risk score was generated, prove a causal mechanism or predict a guaranteed project outcome. Future evaluation should prioritise adjudicated source-level claim checks, risk-specific retrieval benchmarks, more representative holdouts and verification of timestamp eligibility.

9. FINDINGS AND DISCUSSION

The project demonstrates the integration of temporal knowledge representation, machine learning and evidence-based investigation for enterprise project risk analysis. By combining historical Jira records

with supplementary requirement documents and meeting transcripts, the system constructs a connected representation of project activities and their relationships over time.

The experimental results show that predictive performance varies across risk categories. Schedule Risk achieved comparatively stronger results, while Scope and Dependency Risks remain more challenging due to differences in project behaviour and the availability of relevant historical signals. These findings highlight the importance of risk-specific feature engineering and evaluation.

The Agentic GraphRAG component demonstrates how predicted risks can be investigated using connected historical evidence. Event-focused retrieval improves access to relevant project activities, while source references and temporal checks support traceability. However, the evaluation also shows that retrieving relevant evidence and generating factually supported explanations are separate challenges.

Overall, the project establishes a working approach for connecting project risk prediction with evidence-based investigation. Future improvements can focus on expanding risk-specific evidence retrieval, improving temporal event coverage, strengthening prediction across different projects and introducing more comprehensive verification of generated explanations.

References and internal project materials

1. The Public Jira Dataset. Zenodo, record 15719919. [https://zenodo.org/records/15719919].

2. Group 14. 2026. IRS Proposal Presentation. 30 September 2026. Internal project material.

3. Group 14. 2026. GC1 Promotional Presentation. October 2026. Internal project material.

4. Group 14. 2026. Jira PRD and Meeting Generation; Dataset Download. Python implementation files. Internal project material.

5. Group 14. 2026. Entity Extraction; Entity Resolution; Neo4j Temporal Knowledge Graph. Python implementation files. Internal project material.

6. Group 14. 2026. Temporal Knowledge Graph Teammate Handover. October 2026. Internal project material.

7. Group 14. 2026. GC1 Phase 2 Teammate Handover. 12 October 2026. Internal project material.

8. Group 14. 2026. Three-Phase Evaluation Team Summary. October 2026. Internal project material.

FILING REF:

PROJECT PROPOSAL

Date of proposal: 12.09.2026

Project Title: Predictive Enterprise Project Risk Analysis using Temporal Knowledge Graphs

Group ID (As Enrolled in Canvas Class Groups): Group 14

Group Members (name , Student ID):

AKANSHA SAJIMON - A0352134N
CHO LAQSHYA - A0357002R
RAJAMANICKAM ABIRAMI - A0354513J
RAVIKUMAR PRITHIKA - A0354287U

Sponsor/Client: *None*

Background/Aims/Statement:

Enterprise projects rarely fail without warning. The warning signs often appear weeks or months earlier through changing requirements, unresolved dependencies, repeated blockers, conflicting decisions, missed commitments, and concerns raised in meetings.

Yet these signals are scattered across tickets, documents, conversations, and teams, making it difficult for decision-makers to see the larger pattern forming. By the time traditional project indicators clearly show that something is wrong, deadlines may already be at risk, budgets may have been consumed, and critical decisions may be expensive to reverse.

This creates a fundamental visibility gap for enterprises. They can track what has happened, but they struggle to recognize what is likely to happen next and understand the chain of decisions and events driving that risk. The result is a reactive cycle where organizations investigate failures after the damage has occurred instead of identifying the conditions that lead to failure while there is still time to intervene.

There is a need for a predictive system that can continuously learn from the evolving history of a project, recognize early patterns of failure, and warn teams when risk begins to accumulate. However, predicting risk alone is not enough. Enterprise decisions are interconnected, and a single requirement change can trigger a chain of new dependencies, technical blockers, delayed milestones, and downstream decisions that may span multiple teams and systems.

Representing this evolving project knowledge as a temporal graph makes these connections explicit and allows the system to reason across decisions, requirements, dependencies, risks, and events that would otherwise remain fragmented. By combining graph-based project intelligence with predictive modelling, the proposed solution aims to identify not only which projects are moving towards failure, but also the sequence of decisions and events that is driving that risk

Project Descriptions:

Predictive Enterprise Project Risk Analysis is an AI-driven project intelligence system that transforms fragmented enterprise project data into an evolving model of project health, risk, and impact.

The system brings together information from Jira tickets, Microsoft Teams meeting transcripts and notes, and Product Requirements Documents to create a unified view of how a project develops over time. Rather than analysing each source independently, it connects information across these systems to capture how requirements, decisions, dependencies, blockers, milestones, people, and teams influence one another throughout the project lifecycle.

The architecture begins with a data ingestion and preprocessing layer that collects project artifacts, preserves their source and temporal metadata, and divides them into meaningful units for analysis. This ensures that extracted knowledge can always be traced back to when it occurred and where it originated.

A specialised LLM-based knowledge extraction layer then converts this unstructured information into structured entities, relationships, decisions, risks, dependencies, and project events. Schema-constrained extraction is used to maintain consistency across different enterprise sources, while entity resolution connects references to the same requirement, system, feature, person, or team across multiple documents.

The extracted knowledge is standardised using Google Open Knowledge Format and materialised as a temporal Knowledge Graph. This graph acts as the central intelligence layer of the system by connecting information that would otherwise remain isolated across enterprise tools. It also preserves how these relationships change over time, allowing the system to reconstruct the state of a project at different points in its lifecycle.

Graph analytics and temporal feature engineering are then applied to the evolving Knowledge Graph to identify early indicators of project instability. Signals such as requirement volatility, unresolved blockers, dependency complexity, scope growth, decision reversals, ownership gaps, and cross-team dependencies are converted into measurable project risk features.

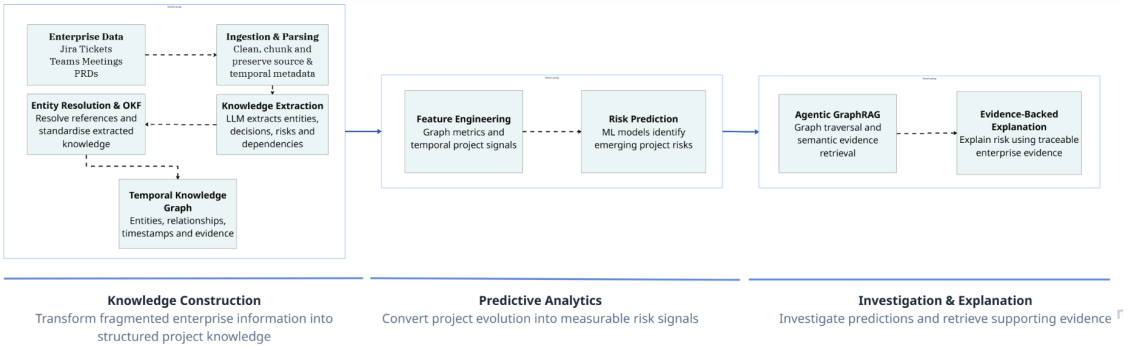
These features are provided to lightweight machine-learning models that estimate Schedule Risk, Scope Risk, Dependency Risk, and Decision Risk. By evaluating these signals across successive project states, the system is designed to identify when risk begins to accumulate rather than waiting for conventional indicators such as missed deadlines or failed milestones.

Prediction is followed by an investigation layer built using Agentic GraphRAG. When a risk is identified, investigation agents traverse the Knowledge Graph to discover the decisions, requirements, dependencies, blockers, and events most strongly associated with that risk. Hybrid retrieval combines graph traversal with semantic retrieval to recover supporting evidence from the original Jira tickets, meeting discussions, and requirement documents.

The final output connects prediction with explanation. Instead of simply reporting that a project is at high risk, the system can identify the sequence of events contributing to that prediction, show how those events are connected, and provide the original enterprise evidence supporting the analysis.

The resulting architecture creates a continuous intelligence pipeline from raw enterprise information to structured knowledge, from structured knowledge to predictive signals, and from predictive signals to evidence-backed investigation.

Stage	Model / Technique	Role
Knowledge extraction	Gemini 2.5 Flash or GPT-5 mini	Extract requirements, decisions, risks, blockers, dependencies, people, milestones and relations using structured output
Entity resolution	Embedding model + LLM verification	Determine whether references such as "Payment Service" and "Payments API" refer to the same entity
Knowledge representation	Google OKF	Standardise extracted knowledge and preserve provenance
Knowledge storage	Neo4j Temporal KG	Store entities, relationships, timestamps and evidence links
Feature generation	Graph algorithms + temporal statistics	Calculate centrality, dependency complexity, blocker age, requirement volatility, decision reversals and similar signals
Risk prediction	XGBoost	Predict Schedule, Scope, Dependency and Decision Risk
Prediction explanation	SHAP	Explain which graph and temporal features drove each risk prediction
Semantic retrieval	Embedding model	Find relevant Jira, Teams and PRD evidence
Investigation	LLM + LangGraph	Plan graph traversal and investigate predicted risks
Final explanation	LLM grounded on retrieved evidence	Produce the human-readable explanation with supporting evidence



Evaluation:

The proposed system will be evaluated at both task and system levels to assess the performance of individual components and the effectiveness of the complete pipeline. At the task level, knowledge extraction will be evaluated using Precision, Recall, and F1-score for entity and relationship extraction, while risk prediction will be assessed using F1-score, PR-AUC, and AUROC. GraphRAG retrieval will be evaluated using Precision@K, Recall@K, and MRR, with Faithfulness and Citation Accuracy used to measure the quality of evidence-grounded explanations. At the system level, Failure Prediction Precision, Recall, and F1-score will measure overall predictive performance. Root-Cause Identification F1-score, Evidence Coverage, and Unsupported Claim Rate will evaluate the reliability and explainability of investigations. Finally, end-to-end response latency and computational cost will be measured to assess the efficiency and practical deployability of the system.

Appendix: Mapping of System Functionalities to Modular Courses

Project Title: Predictive Enterprise Project Risk Analysis using Temporal Knowledge Graphs

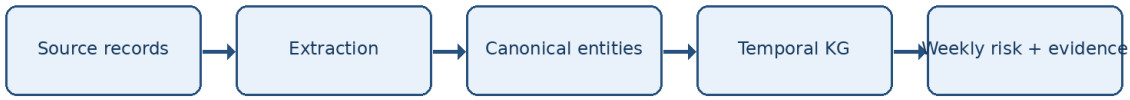
The mapping identifies how implemented project functions apply concepts taught in Machine Reasoning (MR), Reasoning Systems (RS) and Cognitive Systems (CGS). A function may draw on more than one module.

System functionality	Module	Related lecture concepts	Project application
Public Jira, PRD and meeting processing	CGS	Natural-language cognition and information processing	Extracts project concepts from free text using GLiNER alongside structured Jira parsing.
Entity resolution and temporal knowledge graph	MR / RS	Knowledge representation; knowledge graphs; graph-based reasoning	Connects issues, requirements, people and timestamped events using canonical entities and typed relationships.
Point-in-time project reconstruction	RS / MR	Data layer for reasoning; knowledge-based inference	Builds weekly historical states using time-valid facts and relationships.
Risk feature engineering and classification	MR / RS	Inductive reasoning; supervised classification; data-driven reasoning	Converts graph and workflow history into 34 features; applies TabPFN and CatBoost to three risk targets.
Risk explanation with SHAP	MR / RS	Explainability; explainable hybrid reasoning systems	Identifies input features contributing to selected risk scores.
Graph and hybrid evidence retrieval	MR / RS	Search and graph traversal; graph-based retrieval and augmentation	Uses temporal graph traversal, FAISS, BM25, rank fusion and reranking to select relevant historical records.
LLM-based investigation	CGS / RS	Natural-language cognition; reasoning with LLMs; hybrid reasoning architectures	Qwen produces structured, cited observations from checkpoint-eligible evidence.
Evidence and citation checks	RS / CGS	Inference validation; knowledge use and reasoning safeguards	Checks provenance, time eligibility and citation references; allows inconclusive findings.

Appendix — Installation and User Guide

GC1: Predictive Enterprise Project Risk Analysis using Temporal Knowledge Graphs

Purpose: reproduce the research prototype from its supplied Google Colab notebooks and saved project artefacts, inspect historical risk alerts, and review source-linked investigation reports. This is a notebook-based research workflow, not a packaged production application.



1. System overview

The prototype combines public Jira issue histories with generated, temporally aligned PRDs and meeting transcripts. Hybrid extraction produces structured JSONL; entity resolution creates canonical records; temporal graph exports support Neo4j; weekly snapshots produce risk features; TabPFN 2.5 and CatBoost estimate three risk types; SHAP, retrieval and local Qwen support investigation.

2. Prerequisites

Component	Requirement / notes
Execution	Google account with Google Drive and Google Colab; Python notebook execution. A GPU-backed Colab runtime is used for the local Qwen investigation.
Storage	A Drive folder /content/drive/MyDrive/GC1 containing the project dataset and intermediate artefacts. Adjust notebook paths if your Drive layout differs.
Software	Python libraries are installed or imported in the relevant notebook cells. Neo4j is optional for inspecting the imported graph; portable graph exports can be examined without a running database.
Input material	Public Jira archive, selected instance manifests, generated PRD/meeting files, and/or previously saved intermediate outputs.
Models	GLiNER medium-v2.1 for semantic extraction; TabPFN 2.5 and CatBoost for prediction; Qwen2.5-3B-Instruct for local report generation.

Important: The notebooks contain stage-specific configuration and Drive paths. There is no verified single-command installer, pinned environment lockfile or unified web application in the supplied materials.

3. Notebook execution order

Stage	Notebook / action	Expected output or checkpoint
1 — Source setup	Download_dataset.ipynb; Inspect and Select Projects JIRA.ipynb	Public Jira archive and selected repository/instance records; inspect manifests and sample counts.
2 — Aligned artefacts	Jira_PRD_Meeting_Generation_Stage_Instance_Aware.ipynb	Generated PRDs and meeting transcripts aligned to selected Jira windows; retain official generation manifest.

3 — Extraction	Entity_Extraction.ipynb	Common master JSONL with entities, relationships, events, timestamps and provenance.
4 — Resolution	Entity_Resolution.ipynb	Canonical entity/relationship exports; retain source-mention mappings.
5 — Temporal graph	Neo4j_Temporal_KG_Final.ipynb (or the applicable corrected version)	Portable nodes/edges, Neo4j-ready CSV and import scripts; validate IDs, endpoints and times.
6 — Ground truth	GroundTruth_TemporalSnapshots.ipynb	Eligible weekly snapshots and forward 14-day labels.
7 — Features	Feature_Engineering.ipynb	Numerical snapshot feature table (34 selected predictors in the final experiment).
8 — Prediction	Risk_Model_SHAP_Trigger.ipynb	Three risk scores, SHAP drivers and selected investigation triggers.
9 — Retrieval	Event_Focused_Retrieval.ipynb	Historically eligible, ranked evidence bundles and investigation requests.
10 — Explanation	Agentic_GraphRAG_V5_Full_139_Free_GPU.ipynb	Structured Qwen reports, audit CSVs and per-trigger JSON outputs.
11 — Optional professor demo	GC1_Professor_Demo_Live_and_139_Report_Browser.ipynb	Browse all 139 saved V5 reports; optionally generate a fresh Qwen evidence summary. Not required to reproduce the original reports.

4. Setup and execution procedure

Step 1 — Open notebooks

Upload or open the IRS notebook collection in Google Colab. Connect the intended Google Drive account and run each notebook's setup/mount cells. Confirm the root path points to your actual GC1 directory.

Step 2 — Confirm input manifests

Inspect the selected project-instance manifest and official PRD/meeting generation schedule. Do not treat all files present in a directory as official artefacts; use the manifest-selected set.

Step 3 — Execute upstream stages

Run source preparation, extraction and resolution in order, checking each notebook's validation summaries and persisted output before proceeding.

Step 4 — Build and inspect the graph

Generate temporal graph exports; optionally execute the supplied Neo4j import scripts in a configured Neo4j database. Check node/edge totals, missing endpoints and time properties.

Step 5 — Generate historical predictions

Run ground-truth reconstruction and feature engineering before the model notebook. Keep future outcome labels separate from checkpoint-time predictors.

Step 6 — Retrieve and investigate

Generate risk-triggered retrieval inputs. On a GPU runtime run the local Qwen notebook; inspect per-trigger reports and the audit outputs rather than assuming all explanations are verified.

Step 7 — Interactive GraphRAG and Qwen Demonstration

The final demonstration is provided through GC1_Professor_Demo_Live_and_139_Report_Browser.ipynb. This optional notebook supports browsing the 139 previously generated V5 investigation reports and, with a suitable runtime, generating a new evidence summary using the local Qwen2.5-3B-Instruct model.

The demonstration uses three principal saved inputs: test_predictions.parquet (exploratory-test risk predictions), agentic_graphrag_retrieval_queue.jsonl (investigation requests and historically retrieved evidence), and llm_pilot_reports.jsonl (completed V5 investigation reports). Confirm the actual paths in the notebook before running; these filenames identify the expected artefacts, not a guaranteed directory layout.

For the selected ENTESB-I01 case at the 14 October 2020 checkpoint, the saved CatBoost Dependency Risk score was 75.8%, exceeding the configured 9.7% alert threshold. The retrieved evidence includes active Jira blocking dependencies, a newly active dependency and a status transition. The score is a model output, not a calibrated probability of project failure.

The interface displays historical evidence, technical event categories, plain-English summaries and source references. Optional live generation runs Qwen on the saved investigation context; it does not recompute the original risk predictions or perform fresh evidence retrieval. The saved-report browser can be used without rerunning the 139-report generation batch.

Citation-resolution and historical-availability checks verify whether cited evidence IDs exist in the supplied context and were available at the checkpoint. Passing these checks does not independently establish factual accuracy, root cause or causal interpretation.

Required demonstration inputs and outputs

Inputs: test_predictions.parquet; agentic_graphrag_retrieval_queue.jsonl; llm_pilot_reports.jsonl. Outputs: saved report details and citations displayed by the browser; optional new Qwen-generated evidence summary. Preserve the original saved files when testing live generation.

5. User guide: interpreting outputs

User task	Where to look	Interpretation
Inspect graph history	Temporal graph node/edge exports or Neo4j browser	Review linked issues, people, requirements, events and evidence; compare event_time with observed_at.
Inspect risk predictions	Risk model/SHAP/trigger outputs	Separate schedule (TabPFN 2.5), scope (CatBoost) and dependency (CatBoost)

		scores; thresholds are validation-selected.
Inspect an alert	Investigation request / trigger record	Confirm project instance, checkpoint date, risk type, model score and leading SHAP features.
Read investigation	reports_by_trigger/*.json and audit files	Check cited source IDs, timestamp eligibility and conclusion status. "Limited evidence" is not proof of root cause.
Audit historical correctness	Source-to-graph traceability and temporal evaluation notebooks	Check upstream coverage, chronology, valid edge endpoints and whether evidence existed by the checkpoint.
Browse all 139 saved investigations	GC1_Professor_Demo_Live_and_139_Report_Browser.ipynb — saved V5 report browser	Inspect existing report text, case context, citations and conclusion status without rerunning Qwen.
Generate optional live Qwen summary	Same professor demo notebook — live-generation mode	Requires Qwen2.5-3B-Instruct weights and a suitable GPU runtime; reuses saved prediction/retrieval context.

6. Data and output verification

Reference counts for the documented experimental run (not required to match every rerun): 30 project instances; 5,737 Jira ticket-instance records; 30 PRDs; 60 meeting transcripts; 61,392 extracted entity occurrences; 63,305 relationship occurrences; 82,577 temporal event occurrences. The processed temporal graph export reported 96,487 nodes and 164,101 edges. Modelling used 987 eligible weekly checkpoints, 34 predictors and three binary labels.

The full investigation run completed 139 reports: 121 limited-evidence observations and 18 inconclusive. Automated source checks do not establish complete factual accuracy; some claims remained unverified or had citation errors.

7. Troubleshooting and limitations

Symptom	**Recommended check**
FileNotFoundError or empty inputs	Confirm Drive is mounted, notebook root paths match your folder and the prior stage produced its output.
Unexpected project counts	Recheck instance time windows and official PRD/meeting manifest; sampled ticket totals are not identical to eligible instance counts.
Neo4j unavailable	Validate portable CSV/parquet graph exports first; then check Neo4j service/database state and import-script paths.

GPU memory / model loading issue	Use the intended GPU Colab runtime and the notebook's supported loading settings; avoid assuming CPU execution is practical.
Missing or weak explanation	Inspect retrieved evidence, checkpoint eligibility, source IDs and scope-specific events before changing Qwen prompts.
Unsupported causal claim	Treat the report as inconclusive or requiring human review; citations and related facts alone do not prove causation.

8. Reproducibility and responsible use

- Keep original source records, manifests, intermediate exports, notebook versions and audit logs together for reproducibility.
- Never use events observed after a checkpoint as historical model features or as eligible investigation evidence.
- Do not describe model risk scores as calibrated probabilities of project failure without calibration evidence.
- Do not present the prototype as a live enterprise integration or claim that generated explanations are independently verified root causes.
- The saved V5 report browser is inspectable without regenerating all 139 reports. Fresh local Qwen generation requires the model weights, compatible Python dependencies and an adequately provisioned runtime.
- Distinguish three checks: (1) citation resolution verifies evidence IDs, (2) historical availability verifies checkpoint eligibility, and (3) independent source-level factual verification assesses whether claims are supported. The first two do not substitute for the third.

Document basis: GC1 project notebook archive, Group 14 technical report Sections 4–7, and saved full-queue Agentic GraphRAG investigation outputs. Specific dependency versions and a one-command installation procedure are not established by those artefacts.